

August 24, 2025

# Evaluating the Performance of Homogeneous Neural Networks

Internship Report

Lanzeray Etienne, M1 Data Science

Supervisors:

- Mihaly Petreczky, researcher at CRIStAL, CNRS
- Andrey Polyakov, researcher at CRIStAL, INRIA



# Contents

|          |                                                                                         |           |
|----------|-----------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                     | <b>1</b>  |
| 1.1      | Context                                                                                 | 1         |
| 1.2      | Summary of prior research project work                                                  | 1         |
| 1.3      | Internship Objectives                                                                   | 2         |
| <b>2</b> | <b>Dilations, Homogeneity and hNN</b>                                                   | <b>3</b>  |
| 2.1      | Homogeneous Functions and Euler's Definition                                            | 3         |
| 2.2      | Dilations in $\mathbb{R}^n$                                                             | 3         |
| 2.3      | Canonical d-Homogeneous Norm                                                            | 4         |
| 2.4      | Generalized Homogeneous Functions                                                       | 5         |
| 2.5      | Examples of Homogeneous Functions                                                       | 5         |
| 2.6      | Homogeneous Neural Networks                                                             | 6         |
| 2.7      | Universal Approximation Theorem for hNNs                                                | 6         |
| <b>3</b> | <b>Generalization, PAC Learnability, and Rademacher complexity</b>                      | <b>7</b>  |
| 3.1      | Key Concepts and Definitions                                                            | 7         |
| 3.2      | PAC Learnability                                                                        | 7         |
| 3.3      | Uniform Convergence and $\epsilon$ -Representative Samples                              | 8         |
| 3.4      | Rademacher Complexity                                                                   | 8         |
| <b>4</b> | <b>True-Error Properties of hNNs</b>                                                    | <b>9</b>  |
| 4.1      | True error under a dilation-induced covariate shift                                     | 9         |
| 4.2      | Separating radial and angular contributions via a generalized polar change of variables | 10        |
| <b>5</b> | <b>Generalization Bounds for Homogeneous Neural Networks</b>                            | <b>11</b> |
| 5.1      | Rademacher bound for 1 hidden layer Neural Network                                      | 12        |
| 5.2      | Rademacher bound for the hNN prediction class                                           | 13        |
| 5.3      | Extrapolation along dilation orbits: a distribution-shift bound                         | 14        |
| <b>6</b> | <b>Implementation of hNN</b>                                                            | <b>15</b> |
| 6.1      | Key Components                                                                          | 15        |
| 6.2      | Core Building Blocks                                                                    | 15        |
| 6.3      | hNN Forward Map (Definition 5)                                                          | 17        |
| <b>7</b> | <b>Empirical comparison between hNN vs MLP</b>                                          | <b>19</b> |
| 7.1      | Comparison on synthetic data                                                            | 19        |
| 7.1.1    | Data Generation                                                                         | 19        |
| 7.1.2    | Research Project Training Instabilities and Final Resolution                            | 19        |
| 7.1.3    | hNN vs. MLP                                                                             | 20        |
| 7.2      | Pattern Recognition via Homogeneous Features                                            | 24        |
| <b>8</b> | <b>Conclusion</b>                                                                       | <b>27</b> |

## List of Figures

|   |                                                                                                               |    |
|---|---------------------------------------------------------------------------------------------------------------|----|
| 1 | hNN vs. MLP at depth $L = 1$ : the hNN converges while the MLP stagnates around test MSE $\approx 5$ .        | 21 |
| 2 | hNN vs. MLP at depth $L = 2$ : the hNN attains low error early and maintains a clear advantage across widths. | 21 |
| 3 | hNN vs. MLP at depth $L = 4$ : rapid hNN convergence; MLP only catches up at large width and late epochs.     | 22 |
| 4 | Target function vs. hNN vs. MLP, with dilation trajectories shown as black lines.                             | 22 |
| 5 | Target function vs. hNN vs. regularized MLP, with dilation trajectories shown as black lines.                 | 23 |
| 6 | Empirical test error growth under anisotropic dilation for hNN, MLP, and the theoretical scaling law.         | 24 |
| 7 | Characters across different zoom factors.                                                                     | 25 |

## List of Algorithms

|    |                         |    |
|----|-------------------------|----|
| 1  | NORM_P                  | 15 |
| 2  | ISPOSITIVEDEFINITE      | 15 |
| 3  | CHECK_GdP_CONDITIONS    | 15 |
| 4  | DILATION                | 15 |
| 5  | COMPUTEALPHABETA        | 16 |
| 6  | SOLVELOGDNORM_BISECTION | 16 |
| 7  | PROJECTBATCHTODSPHERE   | 17 |
| 8  | SPHEREMLP               | 17 |
| 9  | hNN_FORWARD             | 17 |
| 10 | TRAINHNN                | 17 |
| 11 | TRAINEPOCHHNN           | 18 |
| 12 | EVALUATEHNN             | 18 |

# 1 Introduction

This report documents my Master 1 Data Science internship carried out at CRISAL (CNRS/Inria/Université de Lille) as a direct continuation of my M1 research project on *Homogeneous Neural Networks* (hNNs).

Motivated by the open questions raised during the project and although I not yet produced stable results (training instabilities and implementation questions remained) I was eager to pursue the topic and make the approach work.

I am grateful to **Andrey Polyakov** and **Mihaly Petreczky** for kindly agreeing to supervise me and for hosting the internship within CRISAL.

## 1.1 Context

Deep neural networks (DNNs) have become the cornerstone of modern machine learning applications. However, state-of-the-art architectures often require a substantial number of parameters, leading to challenges in terms of computational cost, memory usage, and generalization ability. One promising approach to mitigating these limitations is to exploit the geometric properties of functions we want to approximate, such as homogeneity.

Homogeneity, introduced by Leonhard Euler in the 18th century, refers to the invariance of functions under specific scaling transformations. This principle has been utilized in various fields, including control theory, physics, and mathematical modeling. Homogeneity inspired a new type of architecture called *Homogeneous Neural Networks* (hNNs) that is proved to be a global approximator of any continuous homogeneous function on  $\mathbb{R}^n$  (Ballesteros et al. 2021). This architecture essentially differs from the use of homogeneous activation functions in ANN (like in (Neyshabur, Salakhutdinov, and Srebro 2015), (Bach 2017)).

Traditional artificial neural networks (ANNs) are known to have strong approximation capabilities, as stated in universal approximation theorems (Cybenko 1989) (Hornik 1990). However, these theorems mainly guarantee approximation on compact domains, limiting their extrapolation abilities when encountering inputs far from the training distribution. *Homogeneous neural networks* (hNNs) address this limitation by enforcing a structural constraint that preserves a dilation symmetry, making it possible to extrapolate global function behavior from local training data.

## 1.2 Summary of prior research project work

In the earlier project, I:

- implemented the hNN architecture of (Andrey Polyakov 2023) in PyTorch, including the  $\mathbf{d}$ -homogeneous norm via a bisection-based solver for  $S = \ln \|x\|_{\mathbf{d}}$ , and the projection  $\pi_{\mathbf{d}}(x) = \mathbf{d}(-S)x$ ;
- tried evaluating on synthetic anisotropic homogeneous targets under diagonal dilation generator  $G_d$ , with inputs sampled on boxes and outputs built from degree-consistent monomials;
- encountered training convergence issues (unstable loss and exploding gradients) which I tried to diagnose: (i) *scale-amplified gradients* due to the  $\|x\|_{\mathbf{d}}^\nu$  output factor (exploding gradients for  $\nu > 0$  and large  $\|x\|_{\mathbf{d}}$ ), and (ii) *projection numerics* (sensitivity in root finding for  $S$  and repeated matrix exponentials);
- tested preliminary mitigations (degree-aware loss weighting, tighter bisection tolerances, closed-form exponentials for diagonal  $G_d$ ), which bounded gradient norms but left loss dynamics unstable.
- Introduces generalization bounds to investigate generalization capabilities of Hnn but didn't have the time to continue on that topic.

### 1.3 Internship Objectives

Building on the prior project, the internship focused on (i) *practical stabilization* of the hNN training pipeline—robust projection, sphere-target learning, regularized sphere MLP, and disciplined optimization—and (ii) *principled evaluation* of extrapolation outside the training convex hull along dilation shells with matched MLP baselines. The generalization analysis is framed through Rademacher complexity specialized to the sphere formulation of hNNs.

Concretely, the internship pursued the following objectives:

- Theoretically investigate generalization bounds using Rademacher complexity tailored to hNNs.
- Empirically evaluate the generalization capabilities of hNNs beyond standard convex hulls;
- Stabilize the training of hNNs on synthetic homogeneous functions;
- Identify and mitigate numerical instabilities (bisection, matrix exponentials);
- Benchmark hNNs against conventional deep neural networks (DNNs) on controlled datasets;

In this report, I first review the theoretical foundations and architecture of homogeneous neural networks (hNNs), following (Andrey Polyakov [2023](#)). I then recall key elements of statistical learning theory and derive hNN-specific generalization results via Rademacher complexity. Finally, I describe a practical implementation (dilation-induced norm, projection by bisection, sphere-MLP) and present experiments that evaluate learnability, extrapolation under dilation shifts, and practical usage.

## 2 Dilations, Homogeneity and hNN

### 2.1 Homogeneous Functions and Euler's Definition

Homogeneous functions were originally introduced by Leonhard Euler in the 18th century in the context of differential equations and mathematical analysis. A function is said to be homogeneous if it exhibits a scaling property under a dilation transformation.

**Definition 1** (Euler's Homogeneous Function). *A function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  is called **standard homogeneous of degree  $\nu$**  if for any scalar  $\lambda > 0$ ,*

$$f(\lambda x) = \lambda^\nu f(x), \quad \forall x \in \mathbb{R}^n. \quad (1)$$

This property implies that multiplying the input by a scalar  $\lambda$  scales the function's output by  $\lambda^\nu$ , where  $\nu$  is called the **degree of homogeneity**.

### 2.2 Dilations in $\mathbb{R}^n$

Euler's classical definition assumes *isotropic scaling* (same factor for all coordinates), which is a special case. In many practical settings (e.g., control theory, physics), scaling transformations are *anisotropic*, meaning that different components of  $x$  may scale at different rates. This leads to the concept of **generalized homogeneity**, which is expressed in terms of **dilations** (generalized scalings).

A family of operators  $\mathbf{d}(s) : \mathbb{R}^n \rightarrow \mathbb{R}^n$  parameterized by  $s \in \mathbb{R}$  is said to form a *one-parameter group* if:

- $\mathbf{d}(0)x = x$ , (identity at  $s = 0$ )
- $(\mathbf{d}(s) \circ \mathbf{d}(t))x = \mathbf{d}(s+t)x$ ,  $\forall x \in \mathbb{R}^n, \forall s, t \in \mathbb{R}$ .

This group of operators is said to be:

- *Continuous* if  $s \mapsto \mathbf{d}(s)x$  is a continuous function for all  $x \in \mathbb{R}^n$ .
- *Linear* if  $\forall s \in \mathbb{R}$ ,  $\mathbf{d}(s)$  is a linear operator (i.e.,  $\mathbf{d}(s) \in \mathbb{R}^{n \times n}$ ).
- *Dilation* (Kawski 1991) if

$$\liminf_{s \rightarrow +\infty} \|\mathbf{d}(s)x\| = +\infty \quad \text{and} \quad \limsup_{s \rightarrow -\infty} \|\mathbf{d}(s)x\| = 0, \quad \forall x \neq 0. \quad (2)$$

Any linear continuous group in  $\mathbb{R}^n$  admits the representation (Pazy 1983):

$$\mathbf{d}(s) = e^{sG_d} = \sum_{j=0}^{\infty} \frac{s^j G_d^j}{j!}, \quad s \in \mathbb{R}. \quad (3)$$

Therefore we can characterize linear dilations by:

**Proposition 1** (Linear Dilation). *A one-parameter family of linear operators  $\mathbf{d}(s) : \mathbb{R}^n \rightarrow \mathbb{R}^n$  is called a **linear dilation** if and only if:*

$$\mathbf{d}(s) = e^{sG_d}, \quad s \in \mathbb{R}, \quad (4)$$

where  $G_d \in \mathbb{R}^{n \times n}$  is anti-Hurwitz matrix<sup>1</sup>.

Matrix  $G_d$  is called the **generator** of the dilation.

---

<sup>1</sup>All eigenvalues have positive real parts

This formulation allows different coordinates to scale at different rates, determined by the eigenvalues of  $G_d$ .

**Definition 2** (Monotonicity and Strict Monotonicity). *A dilation  $d(s)$  in  $\mathbb{R}^n$  is said to be:*

- **Monotone** if the function  $s \mapsto \|d(s)x\|$  is strictly increasing for all  $x \neq 0$ .
- **Strictly Monotone** if there exists a constant  $\beta > 0$  such that:

$$\|d(s)x\| \leq e^{\beta s} \|x\|_P, \quad \forall s \leq 0, \forall x \in \mathbb{R}^n. \quad (5)$$

**Lemma 1.** *A linear continuous dilation in  $\mathbb{R}^n$  is strictly monotone with respect to the weighted Euclidean norm  $\|x\|_P = \sqrt{x^T P x}$  with  $0 \prec P \in \mathbb{R}^{n \times n}$  if and only if<sup>2</sup>:*

$$P G_d + G_d^T P \succ 0, \quad P \succ 0. \quad (6)$$

This definition of monotonicity and lemma are used to define the canonical  $\mathbf{d}$ -homogeneous norm.

### 2.3 Canonical $\mathbf{d}$ -Homogeneous Norm

Given a linear dilation  $\mathbf{d}$ , one can naturally introduce a corresponding "norm"<sup>3</sup>, called the canonical  $\mathbf{d}$ -homogeneous norm, defined implicitly by the dilation itself:

**Definition 3** (Canonical  $\mathbf{d}$ -Homogeneous Norm). *Let a linear dilation  $\mathbf{d}$  in  $\mathbb{R}^n$  be continuous and monotone with respect to a norm  $\|\cdot\|$ . The canonical  $\mathbf{d}$ -homogeneous norm  $\|\cdot\|_{\mathbf{d}} : \mathbb{R}^n \rightarrow [0, +\infty)$  is defined as follows:*

$$\begin{cases} \|x\|_{\mathbf{d}} = e^{s_x}, & \text{where } s_x \in \mathbb{R} : \|\mathbf{d}(-s_x)x\| = 1, \quad x \neq 0. \\ \|0\|_{\mathbf{d}} = 0 \end{cases} \quad (7)$$

**Remark:** For standard dilation  $\mathbf{d}(s) = e^s I_n$ , we recover the original norm, i.e.,  $\|x\|_{\mathbf{d}} = \|x\|_2$ . Generally, the norm  $\|x\|_{\mathbf{d}}$  is implicitly defined as the unique solution of the algebraic equation due to monotonicity of the dilation.

This definition yields several useful properties:

**Lemma 2** (A. Polyakov 2019). *If a linear continuous dilation  $\mathbf{d}$  in  $\mathbb{R}^n$  is monotone with respect to a norm  $\|\cdot\|$ , then the canonical  $\mathbf{d}$ -homogeneous norm  $\|\cdot\|_{\mathbf{d}}$ :*

- 1) is single-valued and continuous on  $\mathbb{R}^n$ ;
- 2) there exist constants  $\sigma_1, \sigma_2 \in \mathcal{K}_{\infty}$  such that:

$$\sigma_1(\|x\|_{\mathbf{d}}) \leq \|x\|_P \leq \sigma_2(\|x\|_{\mathbf{d}}), \quad \forall x \in \mathbb{R}^n; \quad (8)$$

with:

$$\sigma_1(\rho) = \begin{cases} \rho^{\alpha} & \text{if } \rho \leq 1, \\ \rho^{\beta} & \text{if } \rho > 1, \end{cases} \quad \sigma_2(\rho) = \begin{cases} \rho^{\beta} & \text{if } \rho \leq 1, \\ \rho^{\alpha} & \text{if } \rho > 1, \end{cases} \quad (9)$$

where

$$\alpha = \frac{\lambda_{\max} \left( P^{\frac{1}{2}} G_d P^{-\frac{1}{2}} + P^{-\frac{1}{2}} G_d^T P^{\frac{1}{2}} \right)}{2} \geq \beta = \frac{\lambda_{\min} \left( P^{\frac{1}{2}} G_d P^{-\frac{1}{2}} + P^{-\frac{1}{2}} G_d^T P^{\frac{1}{2}} \right)}{2} > 0. \quad (10)$$

<sup>2</sup>Symmetric with positive eigenvalues

<sup>3</sup>Which is not a norm in the traditional sense

- 3) is locally Lipschitz continuous on  $\mathbb{R}^n \setminus \{0\}$ ;
- 4) is continuously differentiable on  $\mathbb{R}^n \setminus \{0\}$  provided  $\|\cdot\|$  is continuously differentiable and  $\mathbf{d}$  is strictly monotone.

Additionally, the projection onto the unit sphere induced by the norm  $\|\cdot\|_{\mathbf{d}}$  is given explicitly by:

$$\pi_{\mathbf{d}}(x) = \mathbf{d}(-\ln \|x\|_{\mathbf{d}})x, \quad (11)$$

and it satisfies:

$$\|\pi_{\mathbf{d}}(x)\| = 1, \quad \forall x \neq 0. \quad (12)$$

Thus,  $\pi_{\mathbf{d}}$  defines a projection from  $\mathbb{R}^n \setminus \{0\}$  onto the  $\mathbf{d}$ -homogeneous unit sphere.

## 2.4 Generalized Homogeneous Functions

Given a dilation operator  $d(s)$ , we extend Euler's definition of homogeneity:

**Definition 4** (Generalized Homogeneous Function). *A function  $h : \mathbb{R}^n \rightarrow \mathbb{R}$  is said to be  $d$ -homogeneous of degree  $\nu \in \mathbb{R}$  if for all  $x \in \mathbb{R}^n$  and  $s \in \mathbb{R}$ ,*

$$h(d(s)x) = e^{\nu s}h(x). \quad (13)$$

This means that  $h(x)$  scales according to the *generalized dilation transformation* rather than the classical isotropic scaling.

## 2.5 Examples of Homogeneous Functions

**Example 1** (Standard Homogeneous Function). *The function  $f(x) = x_1^2 + x_2^2$  is homogeneous of degree 2 with respect to the standard dilation  $d(s)x = e^s x$ , as:*

$$f(d(s)x) = (e^s x_1)^2 + (e^s x_2)^2 = e^{2s} f(x). \quad (14)$$

**Example 2** (Generalized Homogeneous Function). *Consider the function*

$$h(x) = \frac{x_1}{x_2} + \sqrt{x_3}. \quad (15)$$

*It is homogeneous of degree 1 with respect to the anisotropic dilation:*

$$d(s)x = (e^{2s}x_1, e^s x_2, e^{2s}x_3). \quad (16)$$

*since*

$$h(d(s)x) = \frac{(e^{2s}x_1)}{e^s x_2} + \sqrt{e^{2s}x_3} = e^s h(x). \quad (17)$$

**Example 3** (Homogeneous Function in Rigid Body Dynamics). *Following the classical model of a controlled rotational motion of a rigid body in three-dimensional space, the dynamics of the angular velocity of the rigid body is given by:*

$$\dot{\omega}(t) = J^{-1}(-\omega \times J\omega + \tau), \quad t \geq 0, \quad (18)$$

where  $\omega(t) = (\omega_1(t), \omega_2(t), \omega_3(t))^{\top} \in \mathbb{R}^3$  represents the angular velocity in the body frame,  $J = J^{\top} \in \mathbb{R}^{3 \times 3}$  is the inertia matrix, and  $\tau(t) \in \mathbb{R}^3$  is the external momentum (e.g., control input). The operator  $\times$  denotes the vector product, and the angular momentum satisfies:

$$\omega \times J\omega = \begin{bmatrix} 0 & \omega_3 & -\omega_2 \\ -\omega_3 & 0 & \omega_1 \\ \omega_2 & -\omega_1 & 0 \end{bmatrix} J\omega. \quad (19)$$



The function  $h : \mathbb{R}^6 \rightarrow \mathbb{R}^3$  defined by

$$h(\omega, \tau) = -J^{-1}(\omega \times J\omega + \tau), \quad (20)$$

is homogeneous of degree 2 with respect to the generalized dilation:

$$h(e^s \omega, e^{2s} \tau) = e^{2s} h(\omega, \tau), \quad \forall s \in \mathbb{R}, \forall \omega, \tau \in \mathbb{R}^3. \quad (21)$$

## 2.6 Homogeneous Neural Networks

Homogeneous Neural Networks (hNNs) considered in this work are artificial neural networks that incorporate homogeneity constraints into their structure. The fundamental idea is to enforce symmetry with respect to a dilation transformation, making the network inherently suited for approximating homogeneous functions.

**Definition 5** (d-Homogeneous ANN).

- A neural network  $g$  is said to be **d-homogeneous** if it is a **d-homogeneous** function of some degree  $\nu \in \mathbb{R}$ :

$$g(d(s)x) = e^{\nu s} g(x). \quad (22)$$

- An homogeneous Neural Network is defined as:

$$h_\epsilon(x) = \|x\|_{\mathbf{d}}^\nu C \sigma(Ad(-\ln \|x\|_{\mathbf{d}})x + b). \quad (23)$$

where  $A \in \mathbb{R}^{N \times n}$ <sup>4</sup> represents the weight matrix,  $b \in \mathbb{R}^N$  is the bias vector, and  $C \in \mathbb{R}^{1 \times N}$  is the output weight matrix. The activation function  $\sigma$  is applied component-wise.

**Remark:** hNN definition implies its **d-homogeneous**

## 2.7 Universal Approximation Theorem for hNNs

The universal approximation theorem states that a sufficiently large shallow artificial neural network with an activation function  $\sigma$  can approximate any continuous function on a compact domain.

**Theorem 1** (Universal Approximation Theorem, Cybenko 1989). *Let  $\sigma \in C(\mathbb{R}, \mathbb{R})$  be a non-polynomial function, and let  $\Omega \subset \mathbb{R}^n$  be a compact set. If  $g : \mathbb{R}^n \rightarrow \mathbb{R}$  is a continuous function, then for any  $\epsilon > 0$ , there exist  $N \in \mathbb{N}$ ,  $A \in \mathbb{R}^{N \times n}$ ,  $b \in \mathbb{R}^N$  and  $C \in \mathbb{R}^{1 \times N}$  such that:*

$$|g(x) - g_\epsilon(x)| \leq \epsilon, \quad \forall x \in \Omega, \quad (24)$$

where the function  $g_\epsilon$  is given by a single hidden-layer ANN:

$$g_\epsilon(x) = C \sigma(Ax + b). \quad (25)$$

**Remark:** The function  $\sigma$  from the above theorem is known as the *activation function*.

This theorem shows that a Neural Network with one hidden layer is a universal approximator of any continuous function on a compact domain which yields the following result:

**Theorem 2** (Homogeneous Universal Approximation Theorem (Andrey Polyakov 2023)). *Let  $\mathbf{d}$  be a linear continuous dilation in  $\mathbb{R}^n$ . Let  $\|\cdot\|_{\mathbf{d}}$  be an arbitrary (explicit or implicit) **d-homogeneous** norm in  $\mathbb{R}^n$ . Let  $\sigma \in C(\mathbb{R}, \mathbb{R})$  be a non-polynomial function. If  $h \in C(\mathbb{R}^n \setminus \{0\}, \mathbb{R})$  is a **d-homogeneous** function of degree  $\nu \in \mathbb{R}$ , then for any  $\epsilon > 0$ , there exist  $h_\epsilon$  an hNN such that:*

$$|h(x) - h_\epsilon(x)| \leq \epsilon \|x\|_{\mathbf{d}}^\nu, \quad \forall x \in \mathbb{R}^n \setminus \{0\}, \quad (26)$$

**Remark:** The hNN (26) has the structure, which is very similar to the classical ANN (25). The conventional algorithms of ANN training are expected to be applicable to hNN as well.

---

<sup>4</sup>N is the number of hidden units

### 3 Generalization, PAC Learnability, and Rademacher complexity

This section builds upon theoretical foundations from the textbook *Understanding Machine Learning: From Theory to Algorithms* by Shai Shalev-Shwartz and Shai Ben-David (Shalev-Shwartz and Ben-David 2014).

In machine learning, **generalization** refers to a model's ability to perform well on previously unseen data. Various theorems guarantee that certain hypothesis classes (e.g., neural networks under universal approximation assumptions) can approximate any continuous function on a compact domain. However, such guarantees do not specify:

1. How many samples are needed to ensure good approximation on new, unseen data.
2. With what probability over random training sets we can expect a small error.

The **PAC learning framework** addresses exactly these points by introducing notions of accuracy ( $\epsilon$ ) and confidence ( $1-\delta$ ), along with complexity measures (e.g., Rademacher complexity) to capture how large a training set must be for empirical performance to generalize.

Investigating the generalization properties of homogeneous neural networks is particularly interesting, as their inherent structural constraints could yield improved generalization bounds and tighter sample complexity requirements.

#### 3.1 Key Concepts and Definitions

**Definition 6** (Hypothesis Class). A ***hypothesis class***  $\mathcal{H}$  is any set of functions

$$h : \mathcal{X} \rightarrow \mathcal{Y}$$

from which a learning algorithm selects hypotheses to model or predict data.

**Definition 7.** (*Loss Function*)

A **loss function**  $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$  measures the discrepancy between a prediction  $h(x)$  and the true label/target  $y$

**Definition 8.** (*True Risk*)

For a distribution  $\mathcal{D}$  over labeled instances  $(x, y) \in \mathcal{X} \times \mathcal{Y}$ , the **true risk** of a hypothesis  $h$  is:

$$L_{\mathcal{D}}(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(h(x), y)].$$

**Definition 9.** (*Empirical Risk Minimization*)

Given a hypothesis class  $\mathcal{H}$  and a training sample  $S = \{(x_i, y_i)\}_{i=1}^m \sim \mathcal{D}^m$ , the **Empirical Risk Minimization** is a learning algorithm that outputs an **Empirical Risk Minimizer**:

$$h_S \in \arg \min_{h \in \mathcal{H}} L_S(h),$$

#### 3.2 PAC Learnability

A hypothesis class  $\mathcal{H}$  is **PAC learnable** if there is a learning algorithm that, with high probability over the random draw of samples, produces a hypothesis whose true risk is within  $\epsilon$  of the best possible in  $\mathcal{H}$ .

**Definition 10.** (*Agnostic PAC Learnability*)

A hypothesis class  $\mathcal{H}$  is **agnostic PAC learnable** if there is a function

$$m_{\mathcal{H}}(\epsilon, \delta) : (0, 1)^2 \rightarrow \mathbb{N}$$

and a learning algorithm such that, for any distribution  $\mathcal{D}$  over  $(x, y)$ , if we draw  $m \geq m_{\mathcal{H}}(\epsilon, \delta)$  i.i.d. samples from  $\mathcal{D}$ , then with probability at least  $1 - \delta$ :

$$L_{\mathcal{D}}(h) \leq \min_{h' \in \mathcal{H}} L_{\mathcal{D}}(h') + \epsilon,$$

where  $h$  is the hypothesis returned by the algorithm.

**3.3 Uniform Convergence and  $\epsilon$ -Representative Samples**

PAC learnability is closely tied to **uniform convergence**, which ensures empirical risk converges to true risk *uniformly* over all  $h \in \mathcal{H}$ .

**Definition 11.** ( *$\epsilon$ -Representative Sample*)

A sample  $S$  is  $\epsilon$ -representative (relative to  $\mathcal{H}$  and  $\mathcal{D}$ ) if

$$\forall h \in \mathcal{H}, \quad |L_{\mathcal{D}}(h) - L_S(h)| \leq \epsilon.$$

If  $S$  is  $\frac{\epsilon}{2}$ -representative, then *any* empirical risk minimizer (ERM) on  $S$  will also minimize the true risk up to  $\epsilon$ . Formally:

**Lemma 3.** (*ERM and PAC Learnability*)

If  $S$  is  $\epsilon/2$ -representative for  $\mathcal{H}$ , and  $h_S \in \arg \min_{h \in \mathcal{H}} L_S(h)$  is an ERM on  $S$ , then

$$L_{\mathcal{D}}(h_S) \leq \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h) + \epsilon.$$

Thus,  $\epsilon$ -representative samples guarantee PAC Learnability, *provided they occur with high probability over the sample draw*.

**3.4 Rademacher Complexity**

One way to prove that  $\epsilon$ -representative samples occur with high probability is via **Rademacher complexity**.

First let's introduce:

- $\mathcal{F} = \ell \circ \mathcal{H} = \{z = (x, y) \mapsto \ell(h(x), y) \mid h \in \mathcal{H}\}$
- For a sample  $S = (z_1, \dots, z_m)$ ,  $\mathcal{F} \circ S = \{(f(z_1), \dots, f(z_m)) \mid h \in \mathcal{H}\}$

**Definition 12.** (*Rademacher Complexity*)

Given a sample  $S = \{z_1, \dots, z_m\} \subset \mathcal{Z} = \mathcal{X} \times \mathcal{Y}$ , define

$$\mathfrak{R}(\mathcal{F} \circ S) = \frac{1}{m} \mathbb{E}_{\sigma} \left[ \sup_{f \in \mathcal{F}} \sum_{i=1}^m \sigma_i f(z_i) \right],$$

where  $\sigma_1, \dots, \sigma_m$  are i.i.d. Random variables taking values in  $\{-1, +1\}$  with probability  $1/2$ .

**Theorem 3.** (*Generalization via Rademacher Complexity*)

Assuming that  $\forall h \in \mathcal{H}, \forall (x, y) \in \mathcal{Z}, |\ell(h(x), y)| \leq c$

We have that:

$\forall h \in \mathcal{H}$ , with probability at least  $1 - \delta$  over a sample  $S$  of size  $m$ ,

$$|L_{\mathcal{D}}(h) - L_S(h)| \leq 2 \mathfrak{R}(\mathcal{F} \circ S) + 4c \sqrt{\frac{\log(\frac{4}{\delta})}{m}}.$$

**Remark:**

- This kind of inequality bounding the difference between true risk and empirical risk is called a **Generalization Bound**.
- This theorem provides that if the bound:

$$2\mathfrak{R}(\mathcal{F} \circ S) + 4c\sqrt{\frac{\log(4/\delta)}{m}} \leq \frac{\epsilon}{2}$$

holds, then the sample  $S$  is  $\frac{\epsilon}{2}$ -representative. By Lemma 3, this implies that any empirical risk minimizer  $h_S \in \arg \min_{h \in \mathcal{H}} L_S(h)$  satisfies:

$$L_{\mathcal{D}}(h_S) \leq \min_{h \in \mathcal{H}} L_{\mathcal{D}}(h) + \epsilon.$$

Hence, under such conditions, the ERM algorithm ensures agnostic PAC learnability.

This generalization bound highlights the crucial role of **Rademacher complexity**. Specifically, tighter bounds on Rademacher complexity directly yield improved guarantees on the sample complexity required for PAC learnability. Therefore, analyzing and minimizing the Rademacher complexity of hypothesis classes—particularly neural networks—is essential for achieving strong generalization performance.

Now let's investigate some hNN true risk properties before deriving a Rademacher bound for hNN.

## 4 True-Error Properties of hNNs

### Standing notation and assumptions.

Throughout this section we fix:

- A linear dilation group  $\mathbf{d}(s) = e^{sG_d}$  on  $\mathbb{R}^n$  with generator  $G_d$  (anti-Hurwitz).
- A weight  $P \succ 0$  such that  $PG_d + G_d^\top P \succ 0$  (strict monotonicity; cf. Lemma 1).
- The  $P$ -sphere  $S_{\mathbf{d}} := \{x \in \mathbb{R}^n : \|x\|_P = 1\}$  (we keep the subscript  $\mathbf{d}$  to stress its link with the dilation).
- A target  $f$  and an hNN  $h$  that are both  $\mathbf{d}$ -homogeneous of the same degree  $\nu$ , i.e.,  $f(\mathbf{d}(s)x) = e^{\nu s}f(x)$  and  $h(\mathbf{d}(s)x) = e^{\nu s}h(x)$ .

### 4.1 True error under a dilation-induced covariate shift

#### Motivation.

If the data distribution is shifted along dilation orbits (e.g., due to a change of scale along  $\mathbf{d}$ ), how does the true risk of a  $\mathbf{d}$ -homogeneous estimator transform?

Let  $\mathcal{D}$  be a distribution on  $\mathbb{R}^n$  with density  $p$ , and define the shifted distribution  $\mathcal{D}_s$  by pushing forward  $\mathcal{D}$  through  $\mathbf{d}(s)$ , i.e.,

$$X \sim \mathcal{D}, \quad X_s = \mathbf{d}(s)X \sim \mathcal{D}_s.$$

If we apply the dilation to the input space, the new distribution  $\mathcal{D}_s$  has density:

$$\begin{aligned} p_s(x) &= |\det \exp(sG_d)^{-1}| \cdot p(\exp(sG_d)^{-1}x) \\ &= |\det \exp(-sG_d)| \cdot p(\exp(-sG_d)x) \\ &= \exp(-s \operatorname{Tr}(G_d)) \cdot p(\exp(-sG_d)x). \end{aligned}$$

Then, the true error of  $h$  under the shifted distribution  $\mathcal{D}_s$  is:

$$\mathbb{E}_{x \sim \mathcal{D}_s} [(h(x) - f(x))^2] = \int_{\mathbb{R}^n} (h(x) - f(x))^2 p_s(x) dx.$$

Using the change of variables  $x = \exp(sG_d)u$ , we obtain:

$$\begin{aligned}
\mathbb{E}_{x \sim \mathcal{D}_s} [(h(x) - f(x))^2] &= \int_{\mathbb{R}^n} (h(x) - f(x))^2 \exp(-s \operatorname{Tr}(G_d)) p(\exp(-sG_d)x) dx \\
&= \int_{\mathbb{R}^n} (h(\exp(sG_d)u) - f(\exp(sG_d)u))^2 p(u) du \\
&= \int_{\mathbb{R}^n} (e^{\nu s} h(u) - e^{\nu s} f(u))^2 p(u) du \\
&= e^{2\nu s} \int_{\mathbb{R}^n} (h(u) - f(u))^2 p(u) du \\
&= e^{2\nu s} \mathbb{E}_{x \sim \mathcal{D}} [(h(x) - f(x))^2]
\end{aligned}$$

### Takeaway.

Under a pure dilation shift, the squared true error scales by a single factor  $e^{2\nu s}$ :

$$\mathbb{E}_{x \sim \mathcal{D}_s} [(h(x) - f(x))^2] = e^{2\nu s} \mathbb{E}_{x \sim \mathcal{D}} [(h(x) - f(x))^2].$$

This is particularly interesting because it lets us *infer the true risk outside the convex hull of the training data* (along dilation orbits) from the true risk on the training distribution, which itself can be upper-bounded by empirical risk plus a Rademacher term.

### Concrete bound (combine with Thm. 3).

Assume the squared loss is bounded by  $c$  on the region of interest (e.g., via output clipping). Then, with probability at least  $1 - \delta$  over a sample  $S = \{x_i\}_{i=1}^m \sim \mathcal{D}^m$ , for every  $h$  in the class,

$$\mathbb{E}_{x \sim \mathcal{D}_s} [(h(x) - f(x))^2] \leq e^{2\nu s} \left( \underbrace{\frac{1}{m} \sum_{i=1}^m (h(x_i) - f(x_i))^2}_{L_S(h)} + 2\mathfrak{R}(\mathcal{F} \circ S) + 4c \sqrt{\frac{\log(4/\delta)}{m}} \right).$$

Here  $\mathcal{F} = \ell \circ \mathcal{H}$  with  $\ell(u, y) = (u - y)^2$ . Thus, an estimate of in-distribution risk  $L_S(h)$  plus its complexity penalty directly transfers to an *out-of-training-distribution* true risk guarantee.

## 4.2 Separating radial and angular contributions via a generalized polar change of variables

### Idea.

Since  $\mathbf{d}$ -homogeneous models learn only the “angular” part on  $S_{\mathbf{d}}$  and propagate it radially through homogeneity, we can make this precise by switching to generalized polar coordinates  $(s, \theta) \in \mathbb{R} \times S_{\mathbf{d}}$  with the map

$$\Phi : (s, \theta) \mapsto \mathbf{d}(s) \theta = e^{sG_d} \theta,$$

which is a  $C^\infty$  diffeomorphism away from a single dilation ray. This separates the *radial* variable  $s$  from the *angular* variable  $\theta$ .

### Weighted stereographic charts.

Let  $\varphi : S_{\mathbf{d}} \rightarrow \mathcal{S}^{n-1}$  be the  $C^\infty$  diffeomorphism  $\varphi(x) = P^{1/2}x$  with inverse  $\varphi^{-1}(z) = P^{-1/2}z$ . Denote the north/south poles of  $\mathcal{S}^{n-1}$  by  $N = (0, \dots, 0, 1)$ ,  $S = (0, \dots, 0, -1)$ . On the *north* chart  $U_N = S_{\mathbf{d}} \setminus \varphi^{-1}(N)$ , define the stereographic coordinate

$$\text{Stereo}_N(x) = \frac{(P^{1/2}x)_{1:n-1}}{1 - (P^{1/2}x)_n} \in \mathbb{R}^{n-1},$$

with inverse

$$\text{Stereo}_N^{-1}(u) = P^{-1/2} \begin{pmatrix} 2u \\ \frac{1 + \|u\|_2^2}{\|u\|_2^2 - 1} \end{pmatrix}.$$

(Analogous formulas hold for the south chart  $U_S$ .)

On  $U_N$  we use the chart  $\psi_N : \mathbb{R} \times U_N \rightarrow \mathbb{R} \times \mathbb{R}^{n-1}$ ,  $\psi_N(s, \theta) = (s, u)$ ,  $u = \text{Stereo}_N(\theta)$ . Writing

$$f(u) := \begin{pmatrix} \frac{2u}{1 + \|u\|_2^2} \\ \frac{\|u\|_2^2 - 1}{1 + \|u\|_2^2} \end{pmatrix} \quad \text{so that} \quad \text{Stereo}_N^{-1}(u) = P^{-1/2} f(u),$$

the coordinate expression of  $\Phi$  on the north chart is

$$(\Phi \circ \psi_N^{-1})(s, u) = e^{sG_d} P^{-1/2} f(u).$$

**Jacobian.**

Let  $M := P^{1/2} G_d P^{-1/2}$ . Using that  $G_d$  commutes with  $e^{sG_d}$ , we obtain

$$|\det J_{(s,u)}(\Phi \circ \psi_N^{-1})| = e^{s \text{Tr} G_d} \det(P)^{-1/2} |f(u)^\top M f(u)| \left( \frac{2}{1 + \|u\|_2^2} \right)^{n-1}.$$

Here we used that  $f(u) \perp \partial_{u_i} f(u)$  and  $|\det[f(u) \mid Df(u)]| = \left( \frac{2}{1 + \|u\|_2^2} \right)^{n-1}$ .

**Domain and density.**

Fix  $\alpha \in \mathbb{R}$  and consider the truncated dilation cone

$$K_{\alpha,N} := \{\mathbf{d}(s)\theta : s \leq \alpha, \theta \in S_{\mathbf{d}}\} \setminus \{\mathbf{d}(s)P^{-1/2}N : s \in \mathbb{R}\},$$

whose complement of a single ray allows the north chart to cover it. Let  $\mathcal{D}$  be the uniform distribution on  $K_{\alpha,N}$  with density  $p(x) = V_{\alpha,N}^{-1} \chi_{K_{\alpha,N}}(x)$ .

**Radial–angular factorization of the true error.**

Using the change of variables  $x = \Phi \circ \psi_N^{-1}(s, u)$  and  $\mathbf{d}$ -homogeneity of  $h, f$ , we get

$$\begin{aligned} \mathbb{E}_{x \sim \mathcal{D}}[(h(x) - f(x))^2] &= \frac{1}{V_{\alpha,N}} \int_{-\infty}^{\alpha} \int_{\mathbb{R}^{n-1}} (h(e^{sG_d} P^{-1/2} f(u)) - f(e^{sG_d} P^{-1/2} f(u)))^2 |\det J| \, du \, ds \\ &= \underbrace{\frac{1}{V_{\alpha,N} \det(P)^{1/2} (\text{Tr} G_d + 2\nu)}}_{C_{\text{rad}}(\alpha; P, G_d, \nu)} \times \\ &\quad \underbrace{\int_{\mathbb{R}^{n-1}} (h(P^{-1/2} f(u)) - f(P^{-1/2} f(u)))^2 |f(u)^\top M f(u)| \left( \frac{2}{1 + \|u\|_2^2} \right)^{n-1} du}_{\mathcal{E}_{\text{ang}}(h, f; P, G_d)}. \end{aligned}$$

The integral in  $s$  converges whenever  $\text{Tr} G_d + 2\nu > 0$ .

The true error on  $K_{\alpha,N}$  *factorizes* into:

$$\boxed{\mathbb{E}[(h - f)^2] = C_{\text{rad}}(\alpha; P, G_d, \nu) \cdot \mathcal{E}_{\text{ang}}(h, f; P, G_d)}.$$

Thus, for hNNs the learning problem over a dilation cone reduces to: (i) an *angular error* on  $S_{\mathbf{d}}$  with an explicit weight  $w(u) = |f(u)^\top M f(u)| \left( \frac{2}{1 + \|u\|_2^2} \right)^{n-1}$ , and (ii) a purely *radial* scalar multiplier  $C_{\text{rad}}$  that does not depend on the parameters of  $h$ . This formalizes the intuition that hNNs “learn on the sphere” while the radius only rescales the error.

## 5 Generalization Bounds for Homogeneous Neural Networks

**Standing notation for this section.** We adopt the Rademacher complexity notation of Definition 12: for any function class  $\mathcal{U}$  and sample  $S$ ,  $\mathfrak{R}(\mathcal{U} \circ S)$  denotes the empirical Rademacher complexity. We will bound the complexity of *prediction* classes  $\mathcal{G}$  of real-valued functions (e.g., MLPs or hNNs) and then pass to losses via contraction to control  $\mathfrak{R}((\ell \circ \mathcal{G}) \circ S)$ . Inputs are in  $\mathbb{R}^n$ , widths are  $N$ , and  $\phi$  is an activation (ReLU when stated).

## 5.1 Rademacher bound for 1 hidden layer Neural Network

Let the prediction class be

$$\mathcal{G}_{\text{MLP}} = \left\{ g(x) = \langle w, \phi(Ux) \rangle \mid w \in \mathbb{R}^N, \|w\|_2 \leq B_w, U \in \mathbb{R}^{N \times n}, \|u_j\|_2 \leq B_u \ (\forall j \in [N]) \right\},$$

where  $u_j^\top$  is the  $j$ -th row of  $U$ .

(schramm2022mltheory) shows that we have:

**Lemma 4.** Suppose that for any  $\sigma \in \{\pm 1\}^m$  we have  $\sup_\theta \langle \sigma, f_\theta(S) \rangle \geq 0$ . Then

$$\mathbb{E}_\sigma \left[ \sup_\theta |\langle \sigma, f_\theta(S) \rangle| \right] \leq 2 \mathbb{E}_\sigma \left[ \sup_\theta \langle \sigma, f_\theta(S) \rangle \right].$$

**Theorem 4** (Rademacher bound for a single-layer ReLU MLP). Let  $S = (X_i)_{i=1}^m$ ,  $X_i \in \mathbb{R}^n$ , and  $\phi = \text{ReLU}$ . Then

$$\mathfrak{R}(\mathcal{G}_{\text{MLP}} \circ S) \leq \frac{2B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|X_i\|_2^2}. \quad (27)$$

*Proof.*

$$\begin{aligned} \mathfrak{R}(\mathcal{G}_{\text{MLP}} \circ S) &= \frac{1}{m} \mathbb{E}_\sigma \left[ \sup_{\substack{\|w\|_2 \leq B_w \\ \|u_j\|_2 \leq B_u}} \sum_{i=1}^m \sigma_i \langle w, \phi(UX_i) \rangle \right] \\ &= \frac{1}{m} \mathbb{E}_\sigma \left[ \sup_{\substack{\|w\|_2 \leq B_w \\ \|u_j\|_2 \leq B_u}} \left\langle w, \sum_{i=1}^m \sigma_i \phi(UX_i) \right\rangle \right] \\ &= \frac{B_w}{m} \mathbb{E}_\sigma \left[ \sup_{U: \|u_j\|_2 \leq B_u} \left\| \sum_{i=1}^m \sigma_i \phi(UX_i) \right\|_2 \right] \quad (\sup_{\|w\|_2 \leq B_w} \langle w, v \rangle = B_w \|v\|_2) \\ &\leq \frac{B_w}{m} \mathbb{E}_\sigma \left[ \sup_{U: \|u_j\|_2 \leq B_u} \left\| \sum_{i=1}^m \sigma_i \phi(UX_i) \right\|_\infty \right] \quad (\|v\|_2 \leq \sqrt{N} \|v\|_\infty) \\ &= \frac{B_w \sqrt{N}}{m} \mathbb{E}_\sigma \left[ \sup_{U: \|u_j\|_2 \leq B_u} \left| \max_{1 \leq j \leq N} \sum_{i=1}^m \sigma_i \phi(u_j^\top X_i) \right| \right] \\ &= \frac{B_w \sqrt{N}}{m} \mathbb{E}_\sigma \left[ \sup_{\|u\|_2 \leq B_u} \left| \sum_{i=1}^m \sigma_i \phi(u^\top X_i) \right| \right] \\ &\leq \frac{2B_w \sqrt{N}}{m} \mathbb{E}_\sigma \left[ \sup_{\|u\|_2 \leq B_u} \sum_{i=1}^m \sigma_i \phi(u^\top X_i) \right] \quad (\text{Lemma 4}) \\ &\leq \frac{2B_w \sqrt{N}}{m} \mathbb{E}_\sigma \left[ \sup_{\|u\|_2 \leq B_u} \sum_{i=1}^m \sigma_i u^\top X_i \right] \quad (\text{contraction; } \phi \text{ is 1-Lipschitz}) \end{aligned}$$

We now observe that:  $\frac{1}{m} \mathbb{E}_\sigma \left[ \sup_{\|u\|_2 \leq B_u} \sum_{i=1}^m \sigma_i u^\top X_i \right]$  is the Rademacher complexity of the family of linear models:

$$\{x \mapsto \langle u, x \rangle \mid \|u\|_2 \leq B_u\}$$

which is known<sup>5</sup> to be:

$$\leq \frac{B_u}{m} \sqrt{\sum_{i=1}^m \|X_i\|_2^2}$$

□

<sup>5</sup>can be found in (Shalev-Shwartz and Ben-David 2014)

## 5.2 Rademacher bound for the hNN prediction class

Recall the hNN structure (Definition 5): for degree  $\nu$  and a dilation  $\mathbf{d}$ ,

$$h(x) = \|x\|_{\mathbf{d}}^{\nu} \langle w, \phi(U \pi_{\mathbf{d}}(x)) \rangle, \quad \pi_{\mathbf{d}}(x) = \mathbf{d}(-\ln \|x\|_{\mathbf{d}}) x.$$

Define the prediction class

$$\mathcal{G}_{\text{hNN}} = \left\{ g(x) = \|x\|_{\mathbf{d}}^{\nu} \langle w, \phi(U \pi_{\mathbf{d}}(x)) \rangle \mid w \in \mathbb{R}^N, \|w\|_2 \leq B_w, U \in \mathbb{R}^{N \times n}, \|u_j\|_2 \leq B_u \right\}.$$

**Assumption for normalization.** As in Lemma 1, suppose  $\mathbf{d}$  is strictly monotone w.r.t. a norm  $\|\cdot\|$ ; in what follows we take  $P = I$  so that  $\|\cdot\| = \|\cdot\|_2$  and hence  $\|\pi_{\mathbf{d}}(x)\|_2 = 1$  for all  $x \neq 0$ .

**Theorem 5.** Let  $S = (X_i)_{i=1}^m$ ,  $X_i \in \mathbb{R}^n$ , and  $\phi = \text{ReLU}$ . Then

$$\mathfrak{R}(\mathcal{G}_{\text{hNN}} \circ S) \leq \frac{2B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|X_i\|_{\mathbf{d}}^{2\nu}}. \quad (28)$$

*Proof.*

$$\begin{aligned} \mathfrak{R}(\mathcal{G}_{\text{hNN}} \circ S) &= \frac{1}{m} \mathbb{E}_{\sigma} \left[ \sup_{\substack{\|w\|_2 \leq B_w \\ \|u_j\|_2 \leq B_u}} \sum_{i=1}^m \sigma_i \|X_i\|_{\mathbf{d}}^{\nu} \langle w, \phi(U \pi_{\mathbf{d}}(X_i)) \rangle \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[ \sup_{\substack{\|w\|_2 \leq B_w \\ \|u_j\|_2 \leq B_u}} \sum_{i=1}^m \sigma_i \langle w, \|X_i\|_{\mathbf{d}}^{\nu} \phi(U \pi_{\mathbf{d}}(X_i)) \rangle \right] \\ &= \frac{1}{m} \mathbb{E}_{\sigma} \left[ \sup_{\substack{\|w\|_2 \leq B_w \\ \|u_j\|_2 \leq B_u}} \sum_{i=1}^m \sigma_i \langle w, \phi(U (\|X_i\|_{\mathbf{d}}^{\nu} \pi_{\mathbf{d}}(X_i))) \rangle \right] \quad (\phi \text{ is homogeneous of degree } 1) \\ &\leq \frac{2B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|\|X_i\|_{\mathbf{d}}^{\nu} \pi_{\mathbf{d}}(X_i)\|_2^2} \quad (\text{apply Thm. 4 to } S' = (\|X_i\|_{\mathbf{d}}^{\nu} \pi_{\mathbf{d}}(X_i))) \\ &= \frac{2B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|X_i\|_{\mathbf{d}}^{2\nu} \|\pi_{\mathbf{d}}(X_i)\|_2^2} \\ &= \frac{2B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|X_i\|_{\mathbf{d}}^{2\nu}} \quad (\|\pi_{\mathbf{d}}(X_i)\|_2 = 1 \text{ under } P = I). \end{aligned}$$

□

### Takeaway.

Combining Theorem 5 with a standard contraction argument for Lipschitz losses yields an explicit in-distribution generalization bound for hNNs:

Assume outputs are clipped at level  $M$  so that  $|h(x)| \leq M$  and  $|f(x)| \leq M$ . Then the squared loss  $\ell(u, y) = (u - y)^2$  can be written as  $\ell(u, y) = u^2 - 2yu + y^2$ ; on  $[-M, M]$  it is  $4M$ -Lipschitz in  $u$  and vanishes at  $u = 0$  up to a constant independent of  $u$ , hence

$$\mathfrak{R}((\ell \circ \mathcal{G}_{\text{hNN}}) \circ S) \leq 8M \mathfrak{R}(\mathcal{G}_{\text{hNN}} \circ S_X).$$

Plugging this into Theorem 3 and using Theorem 5 gives, with probability at least  $1 - \delta$  over  $S = (X_i, Y_i)_{i=1}^m \sim \mathcal{D}^m$ ,

$$|L_{\mathcal{D}}(h) - L_S(h)| \leq \underbrace{\frac{32 M B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|X_i\|_{\mathbf{d}}^{2\nu}}}_{\text{complexity term}} + \underbrace{16M^2 \sqrt{\frac{\log(4/\delta)}{m}}}_{\text{concentration term}} \quad \forall h \in \mathcal{G}_{\text{hNN}}.$$



In particular, if  $\|X\|_{\mathbf{d}} \leq R$  almost surely, the complexity term scales as  $\frac{32 M B_w B_u \sqrt{N} R^\nu}{\sqrt{m}}$ , so the class  $\mathcal{G}_{\text{hNN}}$  is agnostically PAC learnable and enjoys an  $O(m^{-1/2})$  excess-risk rate with an explicit dependence on the dilation degree  $\nu$ , the width  $N$ , and the weight norms  $B_w, B_u$ .

### 5.3 Extrapolation along dilation orbits: a distribution-shift bound

We now combine the dilation-shift scaling of §4.1 with the hNN generalization bound above to obtain a *true-risk* guarantee on dilated distributions.

**Corollary 1** (Dilation-shift generalization for hNNs). *Let  $h \in \mathcal{G}_{\text{hNN}}$  be an hNN of degree  $\nu$  (matching the target degree), with  $\|w\|_2 \leq B_w$  and  $\|u_j\|_2 \leq B_u$ . Assume clipping at level  $M$  so that  $|h(x)|, |f(x)| \leq M$ . For any  $s \in \mathbb{R}$ , let  $\mathcal{D}_s$  be the pushforward of  $\mathcal{D}$  by the dilation  $\mathbf{d}(s)$ . Then, with probability at least  $1 - \delta$  over the sample  $S = (X_i, Y_i)_{i=1}^m \sim \mathcal{D}^m$ , we have simultaneously for all  $s \in \mathbb{R}$ :*

$$\mathbb{E}_{x \sim \mathcal{D}_s}[(h(x) - f(x))^2] \leq e^{2\nu s} \left( L_S(h) + \frac{32 M B_w B_u \sqrt{N}}{m} \sqrt{\sum_{i=1}^m \|X_i\|_{\mathbf{d}}^{2\nu}} + 16M^2 \sqrt{\frac{\log(4/\delta)}{m}} \right).$$

## 6 Implementation of hNN

This section details the algorithms for the Homogeneous Neural Network (hNN), based on the theoretical concepts introduced in previous sections. I then implemented these algorithms in Python with the PyTorch library due to its efficient handling of tensor computations, GPU acceleration, and automatic differentiation capabilities.

### 6.1 Key Components

- **Dilations** to define anisotropic scaling and the induced  $\mathbf{d}$ -homogeneous norm;
- **Computation of  $\alpha, \beta$**  (Lemma 2) to bound the root-finding interval;
- **Bisection Solver** to compute  $S = \ln \|x\|_{\mathbf{d}}$  and project onto the  $\mathbf{d}$ -sphere;
- **hNN Forward Map** that runs an MLP on the sphere and rescales by  $\|x\|_{\mathbf{d}}'$ .

### 6.2 Core Building Blocks

**Weighted Euclidean norm (Lemma 1)**

---

**Algorithm 1:** NORM\_P

---

**Input:** Vector  $x \in \mathbb{R}^n$ , SPD<sup>a</sup> matrix  $P \in \mathbb{R}^{n \times n}$   
**Output:**  $\|x\|_P = \sqrt{x^\top P x}$   
**return**  $\sqrt{x^\top P x}$

---

<sup>a</sup>Symmetric Definite Positive

**Positive definiteness via Cholesky**

---

**Algorithm 2:** ISPOSITIVEDEFINITE

---

**Input:** Symmetric matrix  $M \in \mathbb{R}^{n \times n}$   
**Output:** **true** iff  $M$  is SPD  
*if  $M$  is not symmetric within tolerance then return false*  
 Attempt a Cholesky factorization  $M = LL^\top$   
*if factorization fails then return false*  
**return true**

---

**Check monotonicity condition (Lemma 1)**

---

**Algorithm 3:** CHECK\_GDP\_CONDITIONS

---

**Input:** Generator  $G_d$ , SPD  $P$   
**Output:** **true** iff conditions of Lemma 1 hold  
*if ISPOSITIVEDEFINITE( $P$ ) = false then return false*  
 Compute  $M \leftarrow PG_d + G_d^\top P$   
*if ISPOSITIVEDEFINITE( $M$ ) = false then return false*  
**return true**

---

**Dilation map  $\mathbf{d}(s) = \exp(sG_d)$**

---

**Algorithm 4:** DILATION

---

**Input:** Generator  $G_d$ , scalar  $s \in \mathbb{R}$ , vector  $x$   
**Output:**  $\mathbf{d}(s)x$   
*if  $G_d$  is diagonal then return  $\text{diag}(e^{sr_1}, \dots, e^{sr_n})x$*   
 Compute matrix exponential  $E \leftarrow \exp(sG_d)$  (stable `expm`/scaling-squaring)  
**return**  $Ex$

---

### Bisection Method & Computation of $\alpha$ and $\beta$

A bisection solver is implemented to efficiently <sup>6</sup> solve the homogeneous norm equation from definition 3:  $s_x = \ln(\|x\|_d)$  such that  $\|d(-s_x)x\| = 1$

This method need bounds which can be found from Lemma 2:

$$\sigma_1(\|x\|_d) \leq \|x\|_P \leq \sigma_2(\|x\|_d), \quad \forall x \in \mathbb{R}^n;$$

$$\sigma_1(\rho) = \begin{cases} \rho^\alpha, & \text{if } \rho \leq 1, \\ \rho^\beta, & \text{if } \rho > 1, \end{cases} \quad \sigma_2(\rho) = \begin{cases} \rho^\beta, & \text{if } \rho \leq 1, \\ \rho^\alpha, & \text{if } \rho > 1. \end{cases}$$

$$\alpha = \frac{\lambda_{\max}\left(P^{\frac{1}{2}}G_dP^{-\frac{1}{2}} + P^{-\frac{1}{2}}G_d^TP^{\frac{1}{2}}\right)}{2} \geq \beta = \frac{\lambda_{\min}\left(P^{\frac{1}{2}}G_dP^{-\frac{1}{2}} + P^{-\frac{1}{2}}G_d^TP^{\frac{1}{2}}\right)}{2} > 0.$$

So we have

$$\min(\|x\|^{1/\alpha}, \|x\|^{1/\beta}) \leq \|x\|_d \leq \max(\|x\|^{1/\alpha}, \|x\|^{1/\beta})$$

$$\implies s_{\min} = \ln(\min(\|x\|^{1/\alpha}, \|x\|^{1/\beta})) \leq s_x \leq s_{\max} = \ln(\max(\|x\|^{1/\alpha}, \|x\|^{1/\beta}))$$

---

#### Algorithm 5: COMPUTEALPHABETA

---

**Input:** Generator  $G_d$ , SPD  $P$

**Output:**  $\alpha \geq \beta > 0$

Eigen-decompose  $P = Q\Lambda Q^\top$  with  $\Lambda = \text{diag}(\lambda_i) > 0$

Compute  $P^{1/2} \leftarrow Q\Lambda^{1/2}Q^\top$ ,  $P^{-1/2} \leftarrow Q\Lambda^{-1/2}Q^\top$

Compute  $S \leftarrow P^{1/2}G_dP^{-1/2} + P^{-1/2}G_d^\top P^{1/2}$

Let  $\lambda_{\max}(S), \lambda_{\min}(S)$  be the extreme eigenvalues of  $S$

$\alpha \leftarrow \frac{1}{2}\lambda_{\max}(S)$ ,  $\beta \leftarrow \frac{1}{2}\lambda_{\min}(S)$

**return**  $(\alpha, \beta)$

---



---

#### Algorithm 6: SOLVELOGDNORM\_BISECTION

---

**Input:**  $G_d, P, x \neq 0, \alpha, \beta$ , tolerance  $\varepsilon > 0$ , max iterations  $T$

**Output:**  $S = \ln \|x\|_d$

Compute  $r \leftarrow \|x\|_P$  via NORM\_P

Set  $a \leftarrow r^{1/\alpha}$ ,  $b \leftarrow r^{1/\beta}$

$S_{\min} \leftarrow \ln(\min(a, b))$ ,  $S_{\max} \leftarrow \ln(\max(a, b))$

**for**  $t = 1, \dots, T$  **do**

$S_{\text{mid}} \leftarrow \frac{1}{2}(S_{\min} + S_{\max})$

    Compute  $z \leftarrow \text{DILATION}(G_d, -S_{\text{mid}}, x)$

    Compute  $q \leftarrow \|z\|_P$

// monotone in  $S$

**if**  $q < 1$  **then**

$S_{\max} \leftarrow S_{\text{mid}}$

**else**

$S_{\min} \leftarrow S_{\text{mid}}$

**end**

**if**  $S_{\max} - S_{\min} < \varepsilon$  **then break**

**end**

**return**  $\frac{1}{2}(S_{\min} + S_{\max})$

---

<sup>6</sup>The first implementation used Newton method but due to convergence issues, M. Polyakov suggested this method

### Batch projection onto the d-sphere

---

**Algorithm 7: PROJECTBATCHTODSPHERE**


---

**Input:**  $G_d, P, \{x_i\}_{i=1}^m, \alpha, \beta, \varepsilon, T$   
**Output:**  $S_i = \ln \|x_i\|_d, \tilde{x}_i = \pi_d(x_i)$

**for**  $i = 1, \dots, m$  **do**  
     $S_i \leftarrow \text{SOLVELOGDNORM\_BISECTION}(G_d, P, x_i, \alpha, \beta, \varepsilon, T)$   
     $\tilde{x}_i \leftarrow \text{DILATION}(G_d, -S_i, x_i)$  //  $\|\tilde{x}_i\|_P = 1$   
**end**  
**return**  $\{(S_i, \tilde{x}_i)\}_{i=1}^m$

---

### 6.3 hNN Forward Map (Definition 5)

#### Sphere MLP (generic, $L$ layers)

---

**Algorithm 8: SPHEREMLP**


---

**Input:** Input  $\tilde{x} \in \mathbb{R}^{d_0}$ ; widths  $(d_1, \dots, d_L)$ ; parameters  $\{W^{(\ell)} \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, b^{(\ell)} \in \mathbb{R}^{d_\ell}\}_{\ell=1}^L$ ; hidden activation  $\phi$ ;  
**Output:**  $u \in \mathbb{R}^{d_L}$

$h \leftarrow \tilde{x}$   
**for**  $\ell = 1$  **to**  $L - 1$  **do**  
     $z \leftarrow W^{(\ell)}h + b^{(\ell)}$   
     $h \leftarrow \phi(z)$  // e.g., tanh/ReLU/GELU  
**end**  
 $u \leftarrow W^{(L)}h + b^{(L)}$   
**return**  $u$

---

#### Direct homogeneous forward

---

**Algorithm 9: HNN\_FORWARD**


---

**Input:** Batch  $X = \{x_i\}_{i=1}^m$ , generator  $G_d$ , SPD  $P$ , degree  $\nu$ , MLP params  $\Theta = \{W^{(\ell)}, b^{(\ell)}\}$ , activation  $\phi$   
**Output:** Predictions  $\{\hat{y}_i\}_{i=1}^m$

Compute  $(\{S_i\}, \{\tilde{x}_i\}) \leftarrow \text{PROJECTBATCHTODSPHERE}(G_d, P, X, \alpha, \beta, \varepsilon, T)$   
**for**  $i = 1$  **to**  $m$  **do**  
     $u_i \leftarrow \text{SPHEREMLP}(\tilde{x}_i; \Theta, \phi)$  // MLP on the d-sphere  
     $\hat{y}_i \leftarrow e^{\nu S_i} u_i$  // rescale by  $\|x_i\|_d^\nu$   
**end**  
**return**  $\{\hat{y}_i\}$

---

#### hNN Training Loop (supervised, MSE-like loss)

---

**Algorithm 10: TRAINHNN**


---

**Input:** Training loader  $\mathcal{D}_{\text{train}}$  yielding mini-batches  $(X, Y)$ ; Test loader  $\mathcal{D}_{\text{test}}$ ; hNN parameters  $\Theta$ ; generator  $G_d$ ; SPD  $P$ ; degree  $\nu$ ; loss function  $\mathcal{L}$  (e.g., MSE); optimizer  $\text{Opt}$  (e.g., Adam) with hyperparameters

**Output:** Sequences of average losses  $\{\ell_{\text{train}}^{(e)}\}_{e=1}^E, \{\ell_{\text{test}}^{(e)}\}_{e=1}^E$ ; updated  $\Theta$

**for**  $e \leftarrow 1$  **to**  $E$  **do**  
     $\ell_{\text{train}}^{(e)} \leftarrow \text{TrainEpochHNN}(\mathcal{D}_{\text{train}}, \Theta, G_d, P, \nu, \mathcal{L}, \text{Opt})$   
     $\ell_{\text{test}}^{(e)} \leftarrow \text{EvaluateHNN}(\mathcal{D}_{\text{test}}, \Theta, G_d, P, \nu, \mathcal{L})$   
**end**  
**return**  $\{\ell_{\text{train}}^{(e)}\}_{e=1}^E, \{\ell_{\text{test}}^{(e)}\}_{e=1}^E, \Theta$

---

---

**Algorithm 11:** TRainePOCHHNN

---

**Input:**  $\mathcal{D}_{\text{train}}$ ;  $\Theta$ ;  $G_d$ ;  $P$ ;  $\nu$ ; loss  $\mathcal{L}$ ; optimizer Opt**Output:** Average training loss  $\ell_{\text{train}}$ Enable training mode for hNN parameters  $\Theta$  $r \leftarrow 0$ 

// running loss accumulator

 $N \leftarrow 0$ 

// number of mini-batches

**foreach**  $(X, Y)$  in  $\mathcal{D}_{\text{train}}$  **do**

ZeroGrad(Opt)

 $\hat{Y} \leftarrow \text{HNN\_FORWARD}(X; G_d, P, \nu, \Theta)$      $L \leftarrow \mathcal{L}(\hat{Y}, Y)$     Backward( $L$ )

Step(Opt)

 $r \leftarrow r + \text{Mean}(L)$ ;     $N \leftarrow N + 1$ **end****return**  $\ell_{\text{train}} \leftarrow r / \max(1, N)$ 

---

---

**Algorithm 12:** EVALUATEHNN

---

**Input:**  $\mathcal{D}_{\text{test}}$ ;  $\Theta$ ;  $G_d$ ;  $P$ ;  $\nu$ ; loss  $\mathcal{L}$ **Output:** Average test loss  $\ell_{\text{test}}$ Enable evaluation mode for hNN parameters  $\Theta$   
updates

// disable dropout/batch-norm

 $r \leftarrow 0$ ; $N \leftarrow 0$ 

Disable gradient computations

**foreach**  $(X, Y)$  in  $\mathcal{D}_{\text{test}}$  **do**     $\hat{Y} \leftarrow \text{HNN\_FORWARD}(X; G_d, P, \nu, \Theta)$      $r \leftarrow r + \text{Mean}(\mathcal{L}(\hat{Y}, Y))$ ;     $N \leftarrow N + 1$ **end**

Enable gradient computations

**return**  $\ell_{\text{test}} \leftarrow r / \max(1, N)$ 

---

## 7 Empirical comparison between hNN vs MLP

### 7.1 Comparison on synthetic data

The goal is to empirically evaluate the performance of a homogeneous neural network (hNN) against a standard MLP in learning to approximate an anisotropic homogeneous function:

$$f_{\text{anisotropic}}(x) = \sum_k c_k |x_1|^{\alpha_{1,k}} |x_2|^{\alpha_{2,k}} \quad (29)$$

The exponents  $\alpha_{1,k}$  and  $\alpha_{2,k}$  satisfy the homogeneity condition:

$$\alpha_{1,k} r_1 + \alpha_{2,k} r_2 = \nu, \quad (30)$$

ensuring that the function is homogeneous under the anisotropic dilation defined by the diagonal generator  $G_d = \text{diag}(r_1, r_2)$ .

#### 7.1.1 Data Generation

Training data is generated synthetically as follows:

- Inputs  $X$  are sampled from a Uniform distribution:

$$X \sim \mathcal{U}([-5, 5]^2), \quad X \in \mathbb{R}^{N \times 2} \quad (31)$$

- Outputs  $y$  are computed from the anisotropic homogeneous function:

$$y_i = f_{\text{anisotropic}}(X_i), \quad i = 1, \dots, N. \quad (32)$$

The training procedure employs:

- Loss function: Mean Squared Error (MSE).
- Optimizer: Adam.

#### 7.1.2 Research Project Training Instabilities and Final Resolution

As outlined in the introduction, I encountered training instabilities during the research project, so my first priority was to diagnose and remedy them.

**Symptoms.** Over a wide range of hyperparameter explorations (learning rate, regularization, optimizer choice, gradient clipping), training of the hNN consistently showed:

- **Exploding gradients** with large and erratic norm spikes;
- **High and unstable loss** with no reliable convergence.

**Hypotheses considered.**

1. **Scaling-induced explosion.** The homogeneous rescaling  $e^{\nu S}$  in  $h(x) = \|x\|_{\mathbf{d}}^{\nu} \text{MLP}(\pi_{\mathbf{d}}(x))$  amplifies gradients by  $\|x\|_{\mathbf{d}}^{\nu}$  during backpropagation, making large- $\|x\|_{\mathbf{d}}$  samples unstable.
2. **Numerical errors in projection.** Instabilities arising from computing  $S = \ln \|x\|_{\mathbf{d}}$  via bisection and from the matrix exponential  $d(s) = \exp(sG_d)$ .

### Mitigations attempted (and their effects).

- **Gradient reweighting.** A weighted loss  $\tilde{\mathcal{L}}(x) = \|x\|_{\mathbf{d}}^{-\nu} (h(x)-y)^2$  (implemented as  $\exp(-\nu S)$  weights) effectively bounded gradient norms (typically in  $[1, 4]$ ) but *did not* stabilize the loss, which continued to oscillate.
- **Numerical tightening.** Tighter bisection tolerances / more iterations and alternative matrix exponential backends (`torch.matrix_exp`, `scipy.linalg.expm`, and closed-form diagonal case) produced negligible change in loss behavior.
- **Precomputed projection.** Treating  $\tilde{x} = \pi_{\mathbf{d}}(x)$  as a fixed data transform (removing bisection and `exp` from the training path) reduced overhead but left loss instability unchanged.

**Interpretation.** Because (i) gradient magnitudes were successfully controlled by the weighted loss, yet (ii) the loss still fluctuated under numerically simpler pipelines, the reason for the instability was unlikely to be an exponential rounding error or a gross projection/gradient scale error.

**Root cause & fix.** The instability ultimately traced back to the **training loop**: the mini-batch loss was *not properly accumulated*, yielding effectively erratic stochastic updates and misleading epoch statistics. Once the loss accumulation across batches was corrected (i.e., averaging the per-batch losses consistently and using that for optimization/logging), training became stable and convergent. In short, what I thought to be optimization/numerical pathologies only were a consequence of my mistake in coding the training loop rather than an inherent issue with the hNN pipeline.

### 7.1.3 hNN vs. MLP

#### Protocol.

- Architectures: depths  $L \in \{1, 2, 4\}$ , widths  $d \in \{16, 32, 64\}$ .
- Data split:  $N_{\text{train}} = 5000$ ,  $N_{\text{test}} = 5000$  (same split for both models).
- Initialization: identical weights per  $(L, d)$  (same seed).
- Training: same batches, schedule, and hyperparameters; loss/optimizer as in the synthetic-data setup above.

#### Results.

Table 1: Test MSE at epochs 100 and 400. Best in each pair is **bold**.

| $L$ | $d$ | HNN@100       | MLP@100       | HNN@400       | MLP@400       |
|-----|-----|---------------|---------------|---------------|---------------|
| 1   | 16  | 6.6685        | <b>5.7901</b> | <b>1.3919</b> | 5.1490        |
| 1   | 32  | 7.2210        | <b>5.7467</b> | <b>1.4064</b> | 5.0438        |
| 1   | 64  | <b>1.6703</b> | 5.2474        | <b>1.4055</b> | 4.9431        |
| 2   | 16  | <b>1.7027</b> | 5.3313        | <b>1.3034</b> | 4.7761        |
| 2   | 32  | <b>1.4072</b> | 5.1450        | <b>1.3523</b> | 4.1455        |
| 2   | 64  | <b>1.3324</b> | 4.9475        | <b>1.1242</b> | 1.4535        |
| 4   | 16  | <b>1.3324</b> | 4.9475        | <b>1.1242</b> | 1.4535        |
| 4   | 32  | <b>1.3654</b> | 4.8335        | 1.0101        | <b>0.8878</b> |
| 4   | 64  | <b>1.2856</b> | 4.6582        | 0.7517        | <b>0.7069</b> |

### Observed trends.

- **Low depth ( $L = 1$ ):** The MLP does not effectively learn the target (test  $\approx 5$  after 400 epochs), whereas the hNN converges to  $\approx 1.4$  across widths.

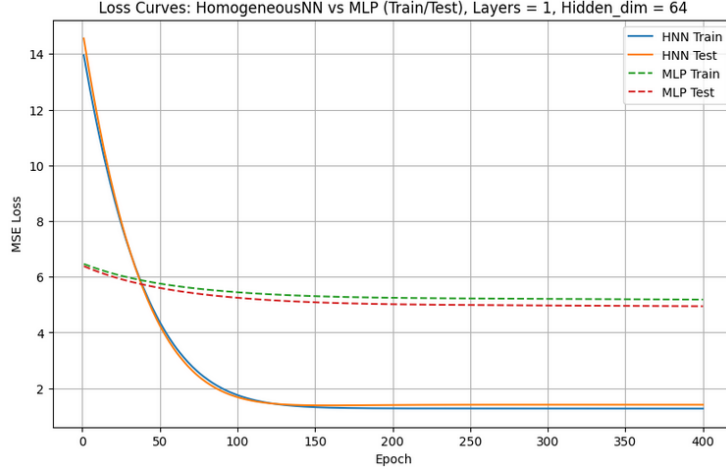


Figure 1: hNN vs. MLP at depth  $L = 1$ : the hNN converges while the MLP stagnates around test MSE  $\approx 5$ .

- **Moderate depth ( $L = 2$ ):** hNN keeps a strong edge at all widths; even at  $d = 64$  the hNN finishes better (1.124 vs. 1.454) and reaches low error far earlier.

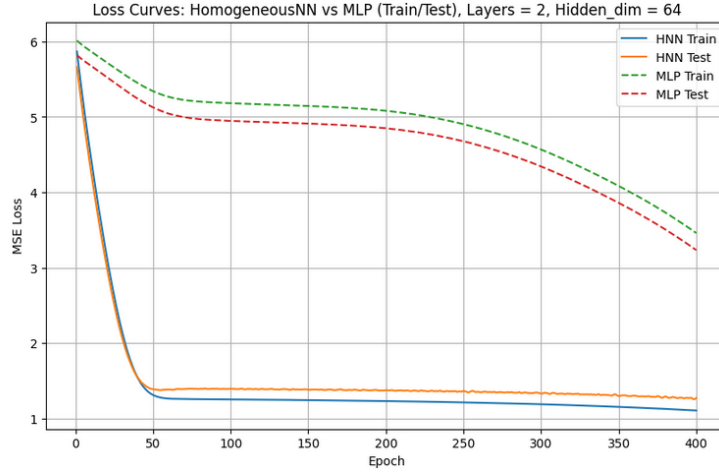


Figure 2: hNN vs. MLP at depth  $L = 2$ : the hNN attains low error early and maintains a clear advantage across widths.

- **Higher depth ( $L = 4$ ):** hNN converges fastest and reaches low error early; with enough width the MLP *eventually* catches up and slightly surpasses final test MSE (0.888 at  $d = 32$ , 0.707 at  $d = 64$ ), but only after long training.



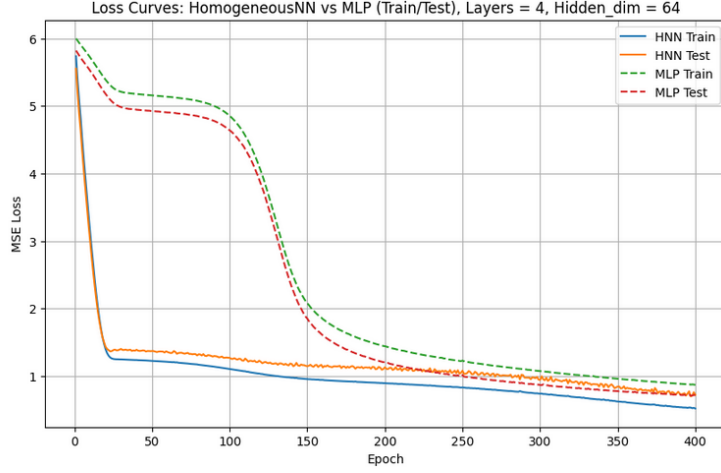


Figure 3: hNN vs. MLP at depth  $L = 4$ : rapid hNN convergence; MLP only catches up at large width and late epochs.

**Takeaway.** For this anisotropic homogeneous target, the hNN’s inductive bias<sup>7</sup> enables faster and more reliable convergence at small and moderate capacities, while the MLP requires deeper/wider architectures and longer training to achieve comparable results. In detail:

- **Low capacity:** With few layers, the MLP scarcely converges, whereas the hNN already reaches low error reliably.
- **Scaling up:** Increasing depth/width accelerates hNN convergence further; only then does the MLP start to converge, still markedly slower.
- **Comparable accuracy:** To reach a similar test error (hNN: 1.39 MSE, MLP: 1.45 MSE at 400 epochs), the hNN needs only *1 layer with 64 hidden units (257 parameters)*, while the MLP requires *4 layers with 16 hidden units (881 parameters)*.
- **High capacity:** Only at the largest depths/widths does the MLP slightly surpass the hNN’s final error, but at the cost of substantially higher capacity and training time.

**MLP yields non homogeneous approximations.**

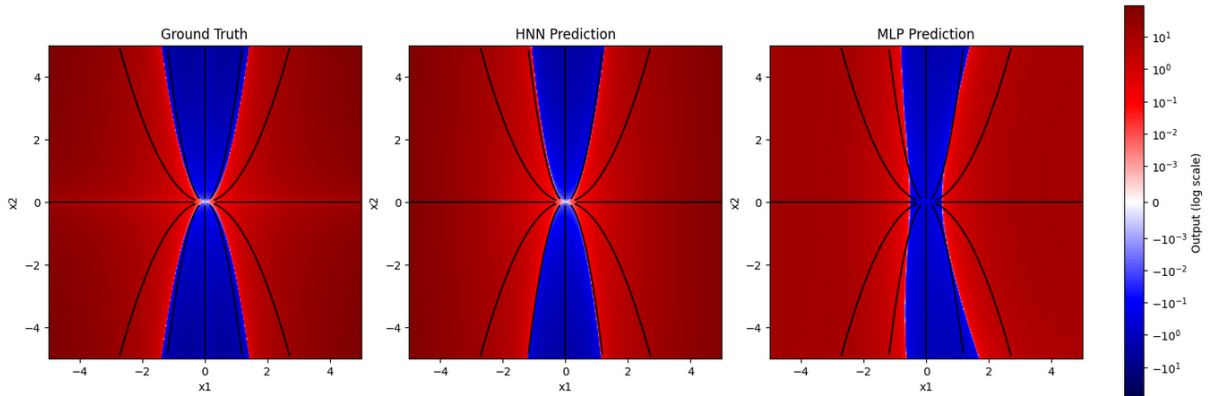


Figure 4: Target function vs. hNN vs. MLP, with dilation trajectories shown as black lines.

Both the target function and the hNN prediction respect the homogeneity property: their level sets align with the dilation trajectories. In contrast, the MLP prediction fails to capture this

<sup>7</sup>Set of assumptions a learning algorithm makes about the target function before seeing any data.

structure. Thus, even when the hNN and MLP achieve comparable test errors, the MLP’s approximations typically break homogeneity, while the hNN preserves it by design.

This homogeneity loss can be problematic in settings where homogeneity is essential. For example, Mr. Polyakov’s work on generalized homogeneous control demonstrates that regulating an LTI plant<sup>8</sup> with an hNN exploiting homogeneity can yield superior performance compared to linear control. Specifically, homogeneity enables faster response times (Zubov 1964; Nakamura, Yamashita, and Nishitani 2002), improved robustness (Ryan 1995; Andrieu, Praly, and Astolfi 2008), and reduced overshoot (A. Polyakov 2020, Ch. 1).

### Homogeneity via regularization loss.

To investigate whether homogeneity could be enforced in a standard MLP, I introduced an additional regularization term which penalizes deviations from the homogeneity constraint in its training objective:

$$\mathcal{L}_{hom} = \left( \|x\|_d^\nu MLP(d(-\ln \|x\|_d) x) - MLP(x) \right)^2,$$

$$\mathcal{L}_{tot} = \mathcal{L}_{MSE} + \alpha \mathcal{L}_{hom}, \quad \alpha > 0 \text{ (regularization parameter).}$$

This encourages the MLP to train toward homogeneous approximations even without explicitly modifying its architecture.

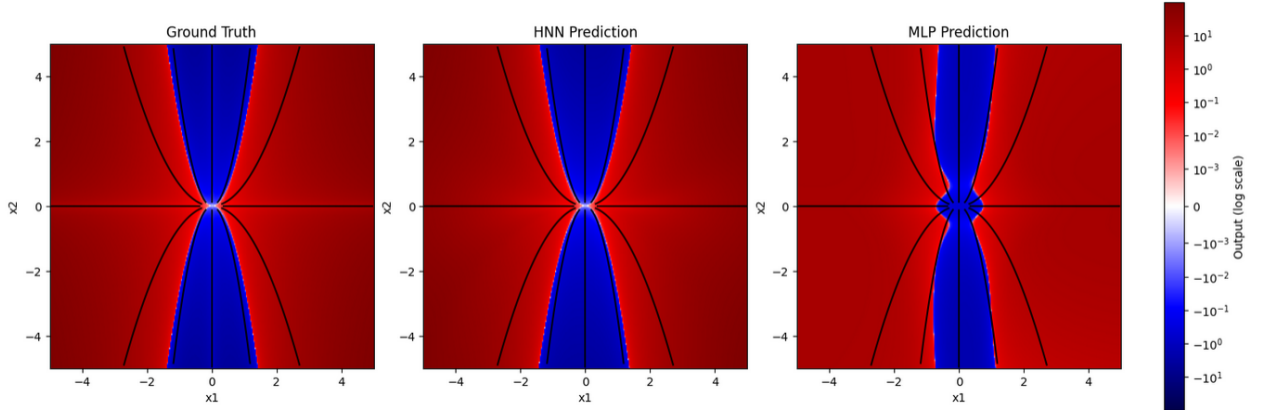


Figure 5: Target function vs. hNN vs. regularized MLP, with dilation trajectories shown as black lines.

Here, the regularized MLP’s level sets align more closely with the dilation trajectories compared to the standard MLP, indicating improved enforcement of homogeneity. However, the approximation remains imperfect and still requires significantly more parameters and training time than the hNN to reach comparable test accuracy.

### Empirical error under test-time dilation shift.

From the analysis of the true error under dilation shift, we established that dilating the input distribution by  $d(s) = \exp(sG_d)$  scales the true error as:

$$\mathbb{E}_{x \sim \mathcal{D}_s} [(h(x) - f(x))^2] = e^{2\nu s} \mathbb{E}_{x \sim \mathcal{D}} [(h(x) - f(x))^2].$$

We now verify this prediction empirically by measuring how the empirical test error evolves when the test set is progressively dilated by  $d(s)$ . This allows us to directly compare the observed error growth with the theoretical scaling law.

<sup>8</sup>an LTI plant is a physical or abstract system whose input–output dynamics can be modeled by a linear, time-invariant system.

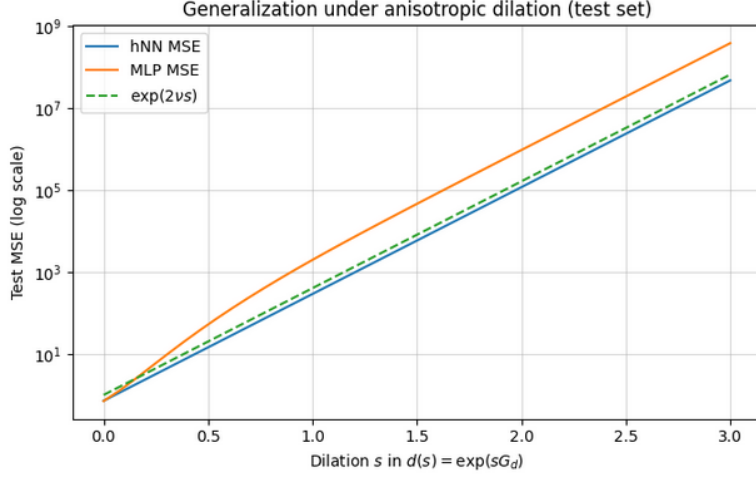


Figure 6: Empirical test error growth under anisotropic dilation for hNN, MLP, and the theoretical scaling law.

We observe that the hNN’s error grows almost exactly according to the theoretical law  $\exp(2\nu s)$ , confirming that it generalizes in a predictable and controlled way under distributional shift. In contrast, the MLP’s error increases more rapidly than  $\exp(2\nu s)$ . This demonstrates that the hNN generalizes significantly better outside the convex hull of its training distribution compared to the MLP.

## 7.2 Pattern Recognition via Homogeneous Features

### Idea.

Following M. Polyakov idea (Andrey Polyakov 2023), we can replace raw pixels by a feature map whose coordinates are homogeneous functionals of the image. Given a grayscale image  $\phi : \mathbb{Z}^2 \rightarrow [0, 255]$  (extended by zero to  $\phi : \mathbb{R}^2 \rightarrow [0, 255]$ ), define

$$x_i(\phi) = \int_{\mathbb{R}^2} b_i(z) \phi(z) dz, \quad i = 1, \dots, n, \quad (33)$$

where  $b_i : \mathbb{R}^2 \rightarrow \mathbb{R}$  is homogeneous of degree  $r_i > 0$ , i.e.  $b_i(e^s z) = e^{r_i s} b_i(z)$ . Under a zoom  $z \mapsto e^s z$  (equivalently  $\phi_s(z) = \phi(e^{-s} z)$ ), a change of variables yields

$$x_i(\phi_s) = e^{(2+r_i)s} x_i(\phi). \quad (34)$$

Thus, the feature vector obeys the anisotropic dilation law

$$x(\phi_s) = d(s) x(\phi), \quad d(s) = \text{diag}(e^{(2+r_1)s}, \dots, e^{(2+r_n)s}) = \exp(s G_d), \quad G_d = \text{diag}(2+r_1, \dots, 2+r_n). \quad (35)$$

A degree-0  $\mathbf{d}$ -homogeneous classifier (our hNN with  $\nu = 0$ ) is then zoom-invariant:

$$h(x(\phi_s)) = h(d(s) x(\phi)) = h(x(\phi)).$$

**Discrete implementation.** For  $100 \times 100$  images, (33) is evaluated via the Riemann sum

$$x_i(\phi) \approx \sum_{k=1}^{100} \sum_{j=1}^{100} b_i(k-50, j-50) \phi(k, j),$$

using a polynomial basis (e.g.  $b_i(z) = z_1^{p_i} z_2^{q_i}$ , so  $r_i = p_i + q_i$ ). Coordinates are centered at the image center and the tensors  $\{b_i(k-50, j-50)\}$  are precomputed.

### Scope and motivation.

We investigate *zoom-invariant pattern recognition* in an *extremely data-efficient* regime: a single example per class at a nominal scale, evaluated under geometric zoom. The goal is to test whether the homogeneous-feature representation (33–35) enables invariance *without* any scale augmentation. To isolate inductive bias rather than data volume, all models are trained only at the nominal scale and then evaluated across zoom factors. We compare:

- an hNN on these features (degree 0, zoom-invariant by design),
- an MLP on the same features (no built-in homogeneity),
- a CNN trained directly on pixels (no scale augmentation).

These MLP/CNN baselines are included purely for reference, under standard practice they typically require multiple examples and augmentation to acquire scale invariance.

### Problem setup and data.

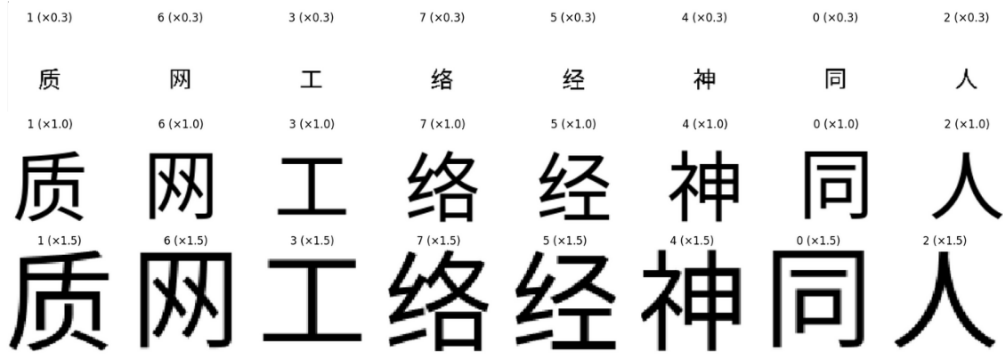
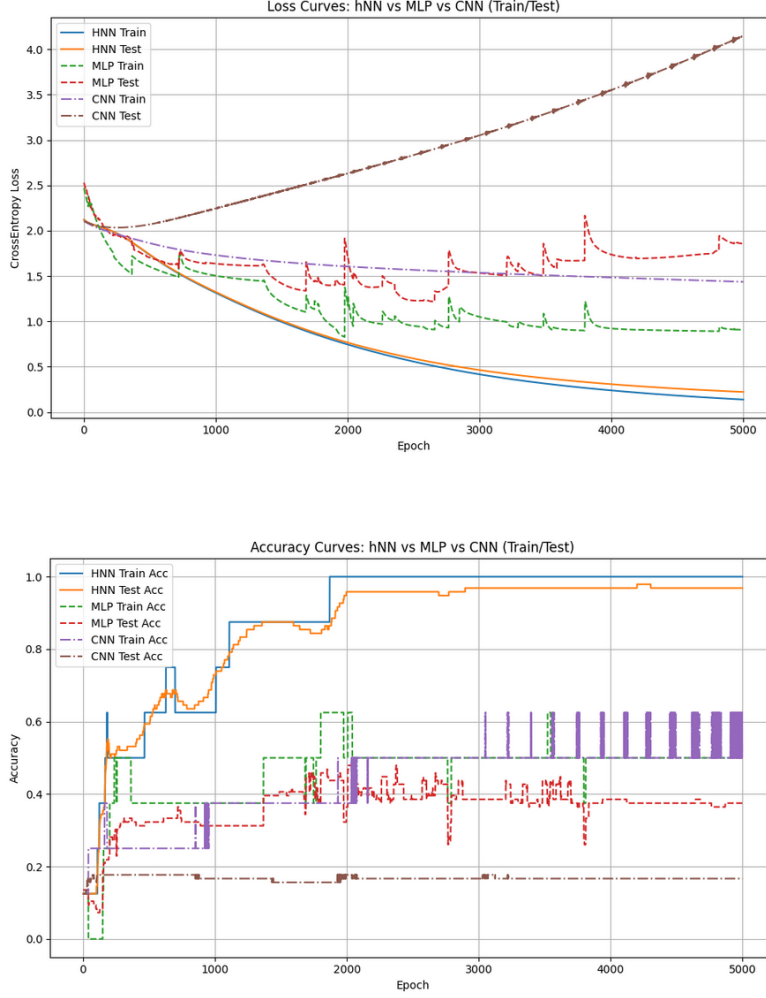


Figure 7: Characters across different zoom factors.

- **Classes:** 8 characters.
- **Nominal size:**  $100 \times 100$  pixels.
- **Zoom factors:**  $\Gamma = \{0.3, 0.4, \dots, 1.5\}$ . Each image is resized to  $(\lfloor 100\gamma \rfloor, \lfloor 100\gamma \rfloor)$ ; for  $\gamma < 1$  it is re-centered and zero-padded to  $100 \times 100$ , for  $\gamma > 1$  it is center-cropped to  $100 \times 100$ .
- **Train/test split:** training uses *only* the nominal scale  $\gamma = 1$ ; testing uses all  $\gamma \in \Gamma \setminus \{1\}$  (optionally with mild additive noise).
- **hNN & MLP Features:**  $x(\phi)$  from (33); under  $s = \ln \gamma$  they scale according to (35).
- **CNN Features:** raw scaled images.
- **Loss:** cross-entropy.
- **Metrics:** nominal scale accuracy (train accuracy), and scaled accuracy (test accuracy).

## Results.



- **hNN:** Train and test losses decrease smoothly; test accuracy rises rapidly and stabilizes near 0.95–1.00 with a small generalization gap. This indicates genuine *zoom invariance* learned from a single example per class, as predicted by the homogeneous-feature construction.
- **MLP:** Despite fitting the training set moderately ( $\approx 0.5$ – $0.6$  accuracy), test accuracy remains low ( $\approx 0.35$ – $0.45$ ) and fluctuates, reflecting the absence of built-in homogeneity: the model does not generalize to unseen zoom factors without augmentation.
- **CNN:** Training loss/accuracy improve slowly, but test accuracy stays near chance ( $\approx 0.15$  for 8 classes), confirming that with only one nominal example per class and no scale augmentation, a CNN does *not* acquire scale invariance.

## Takeaway.

Under the one-shot regime and evaluation across zoom factors, the hNN achieves strong and stable performance, whereas both the MLP and CNN fail to deliver scale-invariant classification (as expected without augmentation). These results suggest that embedding homogeneity via the feature map and a degree-0 hNN yields a practical, *sample-efficient* route to scale-robust recognition well-suited to one-shot settings where collecting multi-scale data or performing heavy augmentation is infeasible.

## 8 Conclusion

On the theory side, I derived an explicit Rademacher-complexity generalization bound for homogeneous neural networks and showed that along dilation orbits (i.e. outside the training convex hull) the squared true error scales as  $e^{2\nu s}$ , yielding principled out-of-distribution guarantees.

Implementation-wise, I built a stable training stack: monotonicity checks,  $(\alpha, \beta)$  computation, bisection-based projection, and a sphere-MLP, and fixed a loss-accumulation bug in the training loop.

Empirically, hNNs converged faster with fewer parameters than MLPs on anisotropic homogeneous targets, matched the predicted  $e^{2\nu s}$  error growth under test-time dilation, and with homogeneous features a degree-0 hNN achieved near scale-invariant one-shot recognition while vanilla MLP/CNNs did not.

### Limitations and perspectives.

The generalization bound, can probably be sharpened (using more recent Rademacher bounds). On the empirical side, testing on other tasks with known scaling laws is a natural next step (like homogeneous control, or adapting it to Homogeneous Galerkin projection to solve EDPs like with Galerkin Neural Networks...).

### Acknowledgments.

I am deeply grateful to my supervisors, **Mihaly Petreczky** (CRISAL, CNRS) and **Andrey Polyakov** (CRISAL, Inria), for their guidance, patience, and insightful discussions. I also thank the CRISAL/Université de Lille environment for a stimulating research setting and the fellow interns and PhD students who provided feedback on both the theory and the implementation.

## References

- Andrieu, V., L. Praly, and A. Astolfi (2008). “Homogeneous approximation, recursive observer design, and output feedback”. In: *SIAM Journal on Control and Optimization* 47.4, pp. 1814–1850.
- Bach, F. (2017). “Breaking the Curse of Dimensionality with Convex Neural Networks”. In: *Journal of Machine Learning Research* 18, pp. 1–53.
- Ballesteros, M. et al. (2021). “Non-parametric identification of homogeneous dynamical systems”. In: *Automatica* 129.7, p. 109600.
- Cybenko, G. (Dec. 1989). “Approximation by Superpositions of a Sigmoidal Function”. In: *Mathematics of Control, Signals and Systems* 2.4, pp. 303–314.
- Hornik, Kurt (Oct. 1990). “Approximation Capabilities of Multilayer Feedforward Networks”. In: Kowski, M. (1991). “Families of dilations and asymptotic stability”. In: *Analysis of Controlled Dynamical Systems*, pp. 285–294.
- Nakamura, H., Y. Yamashita, and H. Nishitani (2002). “Smooth Lyapunov functions for homogeneous differential inclusions”. In: *Proceedings of the 41st SICE Annual Conference*. SICE, pp. 1974–1979.
- Neyshabur, B., R. Salakhutdinov, and N. Srebro (2015). “Path-SGD: Path-Normalized Optimization in Deep Neural Networks”. In: *Advances in Neural Information Processing Systems 28 (NeurIPS 2015)*. The MIT Press.
- Pazy, A. (1983). *Semigroups of Linear Operators and Applications to Partial Differential Equations*. Ed. by Jerrold E. Marsden, L. Sirovich, and F. John. Vol. 44. Applied Mathematical Sciences. New York, NY: Springer.
- Polyakov, A. (2019). “Sliding Mode Control Design Using Canonical Homogeneous Norm”. In: *International Journal of Robust and Nonlinear Control* 29.3, pp. 682–701.
- (2020). *Generalized Homogeneity in Systems and Control*. Cham: Springer.
- Polyakov, Andrey (Nov. 2023). *Homogeneous Artificial Neural Network*. arXiv: [2311.17973](https://arxiv.org/abs/2311.17973) [cs].
- Ryan, E. P. (1995). “Universal stabilization of a class of nonlinear systems with homogeneous vector fields”. In: *Systems & Control Letters* 26, pp. 177–184.
- Shalev-Shwartz, Shai and Shai Ben-David (2014). *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press, pp. I–XVI, 1–397.
- Zubov, V. I. (1964). *Methods of A. M. Lyapunov and Their Applications*. Translated from the Russian (1957). Groningen: Noordhoff.